

Homework 1

Matthew Hutchinson

October 15, 2025

1 Introduction

This assignment explores the dynamic behavior of a mass-spring network using numerical simulation methods covered in MAE 263F: Mechanics of Flexible Structures and Soft Robots. Specifically, we simulate the motion of a 2D spring-mass system (network b) under the influence of gravity using the Implicit Euler and Explicit Euler methods. The model consists of four nodes connected by linear springs, with two nodes fixed and two free to move.

The objective is to:

- Implement a simulator using the Implicit Euler method to evaluate long-term behavior.
- Visualize how the structure evolves over time by plotting network shapes and the displacement of free nodes.
- Experiment with the Explicit Euler method, observe its stability limits, and compare it with the Implicit method.
- Understand the impact of numerical damping introduced by implicit methods and evaluate potential alternatives like the Newmark- β family of time integrators.

By analyzing simulation results and comparing numerical techniques, this assignment highlights the trade-offs between accuracy, stability, and computational efficiency when modeling flexible structures in robotics and structural mechanics.

2 Simulation Setup

This simulation models a 2D spring-mass system defined by a set of nodes connected by linear springs. The system corresponds to spring network b, with four nodes and four springs arranged in a trapezoidal configuration. The dynamics are governed by Newton's second law, with forces arising from spring elongation, external weight due to gravity, and (optionally) damping.

Initial Conditions:

- Only Node 0 and Node 2 are fixed.
- All nodes have equal mass: 1 kg per degree of freedom.
- Spring rest lengths are computed from the initial geometry.
- External forces include gravity applied in the -Y direction.

The system was simulated using:

- Implicit Euler over the interval $t \in [0, 100]$ s
- Explicit Euler over $t \in [0, 10]$ s with varied time steps (Δt)

Simulations were performed using Python with Newton-Raphson iteration for the implicit scheme. The node coordinates and spring connections were loaded from text files.

3 Report Questions

3.1 Pseudocode and code structure

Write pseudocode for your simulator describing its main logic and workflow. Briefly describe the main functions and scripts in your implementation (2–3 sentences each), including their inputs and outputs. Create a simple block diagram showing how the functions or scripts interact (i.e., which one calls which). There is no strict format requirement—use your best judgment to make it understandable to someone with an undergraduate-level engineering background.

3.1.1 Process overview

1. Load initial node positions and spring connections from text files
2. Initialize simulation parameters:
 - Degrees of freedom (DOFs), position, velocity, acceleration
 - Spring rest lengths, spring stiffness, node mass
3. Compute spring rest lengths using Euclidean distance at $t = 0$ and weight (`getFexternal()`)
4. Define time integration parameters (dt , max time, time vector)
5. Loop over time: Use Newton–Raphson iteration (in `myInt()`) to solve the implicit Euler update:
 - Compute total force: inertial + spring + gravity
 - Compute Jacobian of total force (stiffness)
 - Solve linear system for position update
 - Update velocity using $(x_{new} - x_{old})/dt$
 - Plot configuration at specific times
 - Record node positions for output plots
6. After loop ends:
 - Plot and save time histories of x/y positions of free nodes
 - Save snapshots of network shape at specific times

3.1.2 Functions

myInt($t_{new}, x_{old}, u_{old}, \text{free}_{DOF}, \text{stiffness}_{matrix}, \text{index}_{matrix}, m, dt$)

Purpose: Performs one step of implicit Euler integration using Newton-Raphson.

Inputs:

- x_{old}, u_{old} : previous position and velocity
- free_{DOF} : indices of free nodes
- spring stiffness and spring connectivity
- m : mass vector
- dt : time step

Outputs: Updated position x_{new} and velocity u_{new}

Internals:

- Calls `getForceJacobian()` to compute total force and Jacobian
- Solves for Δx using linear solve
- Iterates until convergence

gradEs($x, \text{index}_{matrix}, \text{stiffnesses}$)

Purpose: Computes the gradient of spring energy (i.e., net force)

Inputs: Current positions, spring connectivity, spring constants

Outputs: 4x1 force vector acting on the two nodes

hessEs($x, \text{index}_{matrix}, \text{stiffnesses}$)

Purpose: Computes the Hessian (second derivative) of spring energy

Inputs: Same as gradEs()

Outputs: 4x4 stiffness matrix block for the spring (Jacobian matrix)

getForceJacobian($x_{new}, x_{old}, u_{old}, \text{stiffness}_{matrix}, \text{index}_{matrix}, m, dt, l_k$)

Purpose: Builds the global force vector and Jacobian matrix

Inputs: same as myInt() + spring rest length l_k

Outputs:

- Total force vector: $f = f_{inertia} + f_{spring} + f_{ext}$
- Global stiffness matrix: $J = J_{inertia} + J_{spring} + J_{ext}$

Internals:

- Calculates Inertia: $m\ddot{x}$
- Calls gradEs() to compute the force vector
- Calls hessEs() to compute the stiffness (Jacobian) matrix
- Calls getFexternal() to compute weight of each node
- Solves for updated positions using a linear system

getFexternal(m)

Returns gravitational force $F = mg$ applied on each node

3.1.3 Block Diagram

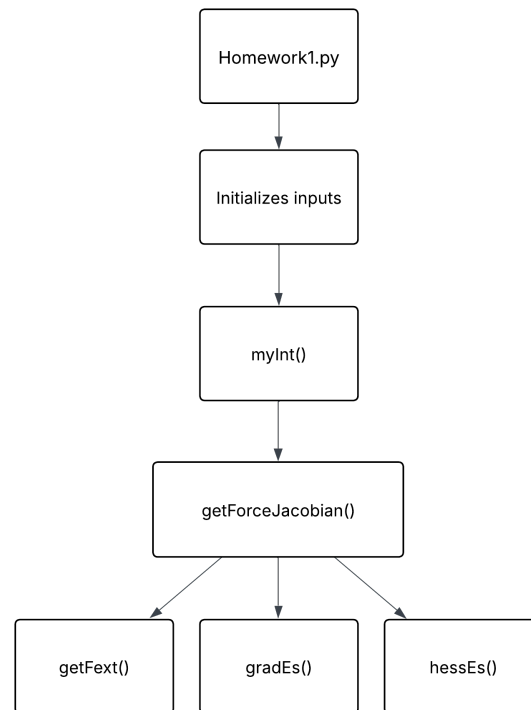


Figure 1: Block Diagram

3.2 How do you choose an appropriate time step size Δt ?

Choosing Δt involves balancing accuracy, stability, and computational efficiency. For explicit methods, Δt must be below a critical threshold (linked to the stiffest spring and smallest mass) to avoid instability. For implicit methods, larger Δt values are possible due to unconditional stability, but extremely small or large Δt can still introduce numerical artifacts or slow convergence.

Observations:

With $\Delta t = 0.5$ s:

The simulation quickly reaches a steady state, but under-resolves the transient dynamics. The system appears to be overdamped even when it is not, and the oscillations are heavily smoothed out. This is computationally efficient but not accurate.

With $\Delta t = 0.1$ s:

The system dynamics are well-resolved: the transient oscillations and damping behaviors are visible, and the motion remains stable. This is an ideal choice for balance between resolution and cost.

With $\Delta t = 0.01$ s:

The plot shows nonphysical behavior—initial amplitude decays correctly, but then grows again over time. This is a sign of numerical instability, likely due to:

- Round-off errors accumulating over a large number of time steps.
- Poor convergence in Newton-Raphson for extremely small updates.
- Insufficient regularization or damping of high-frequency modes.

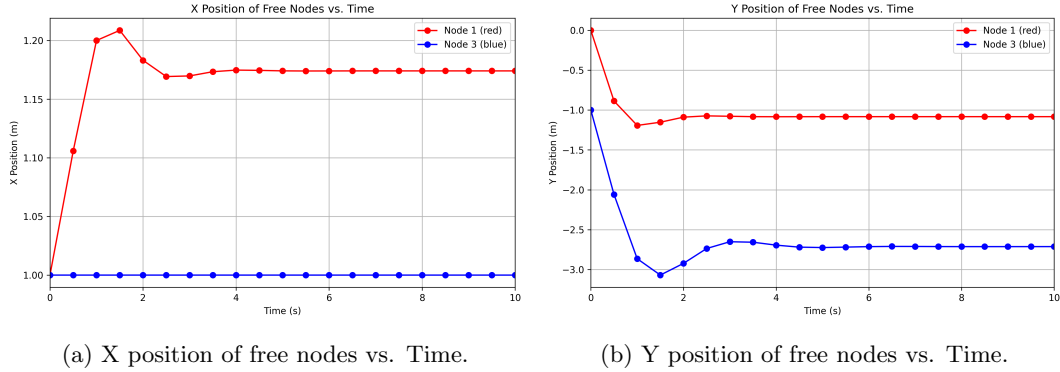
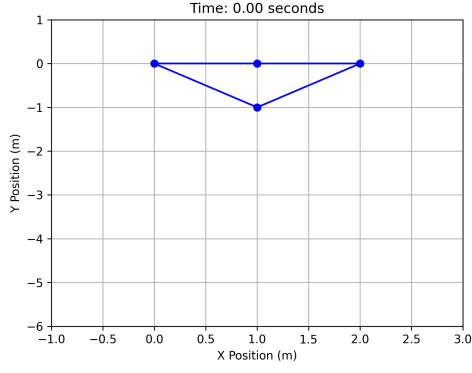
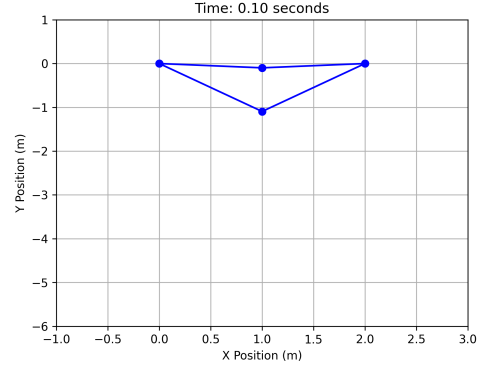


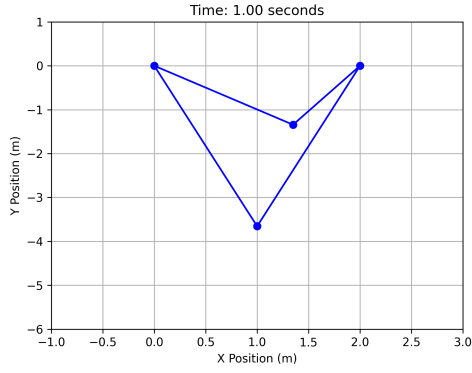
Figure 2: Spring network simulation using $dt = 0.5$ s.



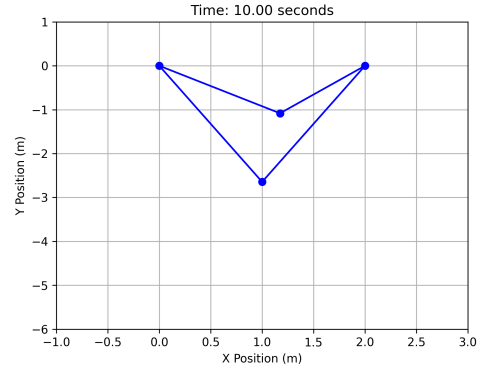
(a) Shape of spring network at $t = 0.0$ s.



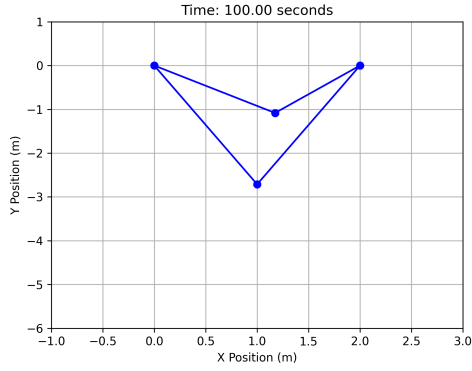
(b) Shape of spring network at $t = 0.1$ s.



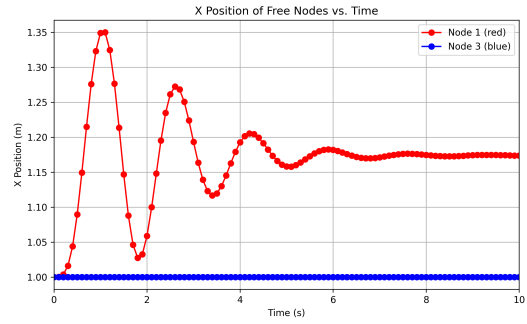
(c) Shape of spring network at $t = 1.0$ s.



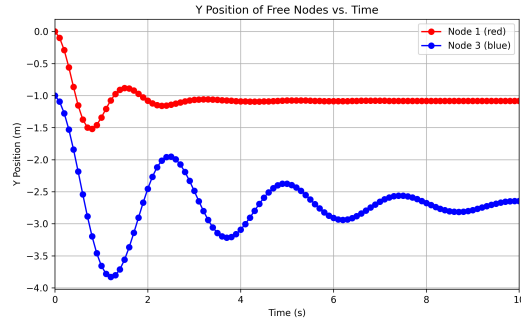
(d) Shape of spring network at $t = 10.0$ s.



(e) Shape of spring network at $t = 100.0$ s.

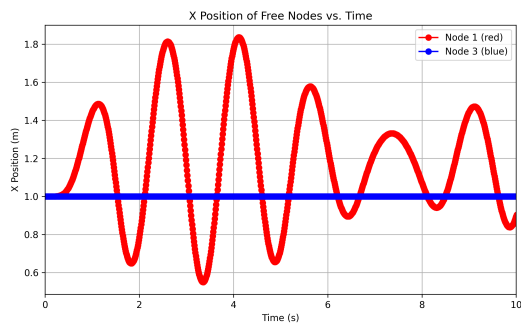


(f) X position of free nodes vs. Time.

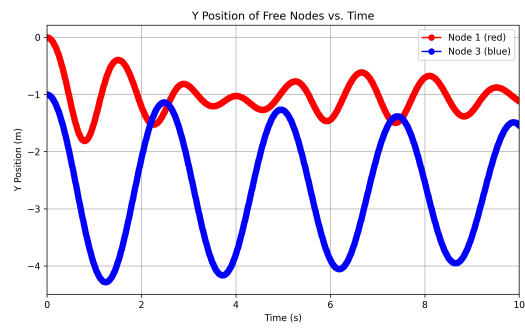


(g) Y position of free nodes vs. Time.

Figure 3: Spring network simulation using $dt = 0.1$ s.



(a) X position of free nodes vs. Time.



(b) Y position of free nodes vs. Time.

Figure 4: Spring network simulation using $dt = 0.01$ s.

3.3 Simulate the spring network using Explicit Euler for $t \in [0, 1]$ s. If it becomes unstable, reduce Δt and try again. If it still fails even at $\Delta t = 10^{-6}$ s, state this in the report. Explain which method (implicit vs. explicit) is preferable for this spring network and why.

I simulated the spring network using the Explicit Euler method over the time interval $t \in [0, 10]$ s. The system includes spring forces, gravity (external weight), and velocity-proportional damping.

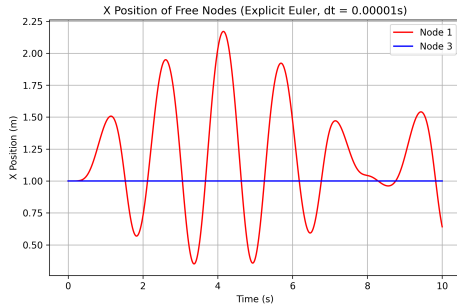
Observations:

- For initial time steps $\Delta t = 10^{-2}$ and $\Delta t = 10^{-3}$, the simulation exhibited unstable behavior—energy accumulated over time, leading to growing oscillations and divergence.
- At $\Delta t = 10^{-5}$, the system still remained unstable.
- Even at $\Delta t = 10^{-6}$, the solution showed non-physical energy growth and failed to converge, although some oscillation suppression was observed.
- I introduced velocity-proportional damping (e.g., $c = 0.5$), which reduced the amplitude of oscillations, but the system still failed to fully stabilize.

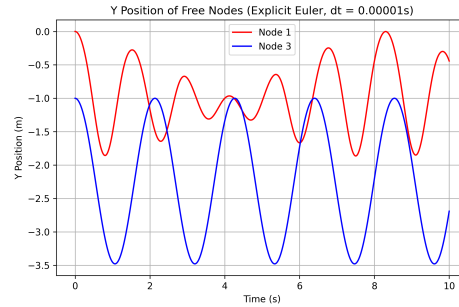
These results indicate that the Explicit Euler method is not suitable for this spring network, even with extremely small time steps and damping.

Method Comparison:

In contrast, the Implicit Euler method was significantly more stable and efficient, even at much larger time steps. This is expected, as implicit methods are unconditionally stable, making them particularly well-suited for stiff systems like spring networks—where large spring constants introduce high-frequency oscillations.

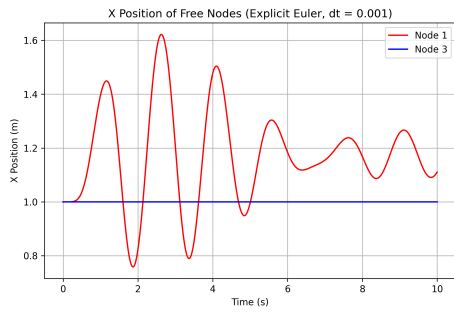


(a) X position of free nodes vs. Time.

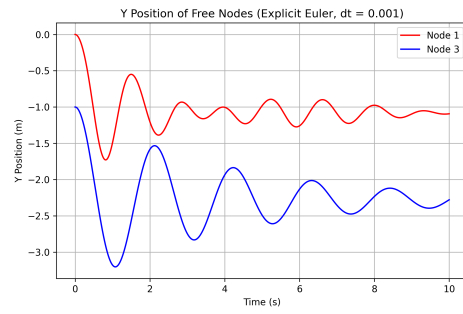


(b) Y position of free nodes vs. Time.

Figure 5: Explicit Euler spring network simulation using $dt = 0.00001$ s with no damping.



(a) X position of free nodes vs. Time.



(b) Y position of free nodes vs. Time.

Figure 6: Explicit Euler spring network simulation using $dt = 0.00001$ s with velocity-proportional damping ($c = 0.5$).

3.4 The simulation with implicit Euler appears to reach a static state (numerical damping). Read about the Newmark- β family of time integrators and explain how such integrators can mitigate such artificial damping.

The Implicit Euler method is a first-order integration scheme known for its unconditional stability. However, it also introduces significant numerical damping, especially in systems with high-frequency modes. This can cause the simulation to artificially settle into a static state, even when physical oscillations should persist.

The Newmark- β family of time integrators offers a more flexible approach that can control or eliminate artificial damping by tuning two parameters: β and γ .

- When $\beta = 1/4$ and $\gamma = 1/2$, the method is unconditionally stable and second-order accurate. This is often used in structural dynamics.
- Unlike Implicit Euler, this choice preserves oscillatory behavior better and avoids excessive damping of low-frequency (physical) modes.
- To introduce targeted numerical damping, higher values of $\beta > 1/4$ and $\gamma > 1/2$ can be used. This preferentially damps high-frequency noise while preserving meaningful system dynamics.

Thus, the Newmark- β method allows us to balance between stability and physical fidelity, providing better control over artificial damping than the overly-dissipative Implicit Euler.